

Task 1

--Q1.1 Edit Distance Levenshtein:

Code :

```
4  import sys
5  sys.path.append('.')
6
7  import csv_loader as csv
8  import measurement as measure
9  from restaurant import restaurant as res
10 import datetime
11
def calc_ed_levenshtein(s1, s2, i=None, j = None, dp = None):
    if dp is None:
        dp = {}

    if i == 0:
        return j

    if j == 0:
        return i

    i = len(s1) if i is None else i
    j = len(s2) if j is None else j
    if (i, j) in dp:
        return dp[(i,j)]

    if s1[i-1] == s2[j-1]:
        dp[(i, j)] = calc_ed_levenshtein(s1, s2, i-1, j-1, dp)
    else:
        dp[(i,j)] = min(
            calc_ed_levenshtein(s1, s2, i-1, j, dp) + 1,
            calc_ed_levenshtein(s1, s2, i, j-1, dp) + 1,
            calc_ed_levenshtein(s1, s2, i-1, j-1, dp) + 1
        )
    return dp[(i, j)]
```

```

def nested_loop_by_ed(csv_path, benchmark_path):
    threshold = 0.98

    restaurants = csv.csv_loader(csv_path)
    results = []
    restaurant1 = res()
    restaurant2 = res()
    id1 = 0
    id2 = 0
    name1 = None
    name2 = None
    start_time = datetime.datetime.now()
    for i in range(0, len(restaurants)):
        restaurant1 = restaurants[i]
        id1 = restaurant1.get_id()
        name1 = restaurant1.get_name()
        for j in range(i + 1, len(restaurants)):
            restaurant2 = restaurants[j]
            id2 = restaurant2.get_id()
            name2 = restaurant2.get_name()
            raw_distance = calc_ed_levenshtein(name1, name2)
            # Normalize edit distance
            max_len = max([len(name1), len(name2)])
            normalized_distance = raw_distance / max_len if max_len > 0 else 0

            # Convert value to similarity score
            sim = 1 - normalized_distance
            if sim >= threshold:

```

```

                # Convert value to similarity score
                sim = 1 - normalized_distance
                if sim >= threshold:
                    results.append(str(id1) + '_' + str(id2))

```

```

end_time = datetime.datetime.now()
time = end_time - start_time
print("Total Time:", round(time.total_seconds() * 1000, 3), 'milliseconds')
print(f"Chosen Threshold levenshtein: {threshold}")
print(f"Total Number of Similar Pairs levenshtein: {len(results)}")

```

```

benchmark = measure.load_benchmark(benchmark_path)
measure.calc_measure(results, benchmark)

```

```

# End of function

```

```

nested_loop_by_ed("/home/s4775476/a2/restaurant.csv", "/home/s4775476/a2/restaurant_pair.csv")

```

Hyper-parameter Analysis:

Total Time: 33339.171 milliseconds
Chosen Threshold levenshtein: 0.1
Total Number of Similar Pairs levenshtein: 270128
Precision= 0.0003887046141088669 , Recall= 0.9905660377358491 , Fmeasure= 0.0007771042873953685

Total Time: 33216.566 milliseconds
Chosen Threshold levenshtein: 0.3
Total Number of Similar Pairs levenshtein: 18817
Precision= 0.005261199978742626 , Recall= 0.9339622641509434 , Fmeasure= 0.010463457168525075

Total Time: 34053.118 milliseconds
Chosen Threshold levenshtein: 0.5
Total Number of Similar Pairs levenshtein: 1364
Precision= 0.06451612903225806 , Recall= 0.8301886792452831 , Fmeasure= 0.11972789115646258

Total Time: 33311.906 milliseconds
Chosen Threshold levenshtein: 0.65
Total Number of Similar Pairs levenshtein: 207
Precision= 0.391304347826087 , Recall= 0.7641509433962265 , Fmeasure= 0.5175718849840256

Total Time: 33119.81 milliseconds
Chosen Threshold levenshtein: 0.7
Total Number of Similar Pairs levenshtein: 141
Precision= 0.5602836879432624 , Recall= 0.7452830188679245 , Fmeasure= 0.6396761133603239

Total Time: 33051.549 milliseconds
Chosen Threshold levenshtein: 0.82
Total Number of Similar Pairs levenshtein: 89
Precision= 0.8764044943820225 , Recall= 0.7358490566037735 , Fmeasure= 0.8

Total Time: 33011.77 milliseconds
Chosen Threshold levenshtein: 0.9
Total Number of Similar Pairs levenshtein: 86
Precision= 0.8953488372093024 , Recall= 0.7264150943396226 , Fmeasure= 0.8020833333333334

Total Time: 33145.962 milliseconds
Chosen Threshold levenshtein: 0.98
Total Number of Similar Pairs levenshtein: 82
Precision= 0.9146341463414634 , Recall= 0.7075471698113207 , Fmeasure= 0.7978723404255319

Note: note as threshold increase , similar pairs decreases. Best threshold is around 0.82 range +- 0.01 because 0.9 threshold similar pair only 86 from 89 only decrease by 3.

--Q1.2 tri-grams (Jaccard Coefficient):

Code : Major reference was taken from nested_loop_by_name_jaccard.py and modified.

```
# task 1.2
import similarity_jaccrd as similarity
import csv_loader as csv
import measurement as measure

from restaurant import restaurant as res
import datetime
```

```
def nested_loop_by_name_jaccard(csv_path, benchmark_path):
    threshold = 0.99
    q = 3

    restaurants = csv.csv_loader(csv_path)
    results = []
    restaurant1 = res()
    restaurant2 = res()
    id1 = 0
    id2 = 0
    name1 = None
    name2 = None
    start_time = datetime.datetime.now()
    for i in range(0, len(restaurants)):
        restaurant1 = restaurants[i]
        id1 = restaurant1.get_id()
        name1 = restaurant1.get_name()
        for j in range(i + 1, len(restaurants)):
            restaurant2 = restaurants[j]
            id2 = restaurant2.get_id()
            name2 = restaurant2.get_name()
            sim = similarity.calc_jaccard(name1, name2, q)
            if sim >= threshold:
                results.append(str(id1) + '_' + str(id2))
```

```

        sim = similarity.calc_jaccard(name1, name2, q)
        if sim >= threshold:
            results.append(str(id1) + '_' + str(id2))

end_time = datetime.datetime.now()
time = end_time - start_time
print("Total Time:", round(time.total_seconds() * 1000, 3), 'milliseconds')
print(f"Chosen Threshold: {threshold}")
print(f"Total Number of Similar Pairs: {len(results)}")
benchmark = measure.load_benchmark(benchmark_path)
measure.calc_measure(results, benchmark)

# End of function

nested_loop_by_name_jaccard("/home/s4775476/a2/restaurant.csv", "/home/s4775476/a2/restaurant_pair.csv")

```

From the Similarity file:

```

def calc_jaccard(str1, str2, q):
    str1_tokens = tokenize(str1, q)
    str2_tokens = tokenize(str2, q)
    total_tokens = str1_tokens + str2_tokens
    total_tokens = list(set(total_tokens))
    return (len(str1_tokens) + len(str2_tokens) - len(total_tokens)) / len(total_tokens)

def tokenize(string, q):
    if q != 0:
        if len(string) < q:
            str_tokens = [string]
        else:
            str_tokens = [string[i:i + q] for i in range(0, len(string) - q + 1, 1)]
        return list(set(str_tokens))
    else:
        str_tokens = string.split(" ")
        return list(set(str_tokens))

```

Hyper-parameter Analysis:

Note: Since Tutor mention q can be set as 3. We will set q = 3.

Total Time: 1375.754 milliseconds

Chosen Threshold: 0.1

Total Number of Similar Pairs: 6151

Precision= 0.016907819866688344 , Recall= 0.9811320754716981 , Fmeasure= 0.03324276809972831

Total Time: 1381.627 milliseconds

Chosen Threshold: 0.35

Total Number of Similar Pairs: 295

Precision= 0.3220338983050847 , Recall= 0.8962264150943396 , Fmeasure= 0.4738154613466334

Total Time: 1375.767 milliseconds

Chosen Threshold: 0.5

Total Number of Similar Pairs: 133

Precision= 0.6541353383458647 , Recall= 0.8207547169811321 , Fmeasure= 0.7280334728033473

Total Time: 1361.559 milliseconds

Chosen Threshold: 0.6

Total Number of Similar Pairs: 100

Precision= 0.82 , Recall= 0.7735849056603774 , Fmeasure= 0.796116504854369

Total Time: 1356.167 milliseconds

Chosen Threshold: 0.75

Total Number of Similar Pairs: 86

Precision= 0.9069767441860465 , Recall= 0.7358490566037735 , Fmeasure= 0.8124999999999999

Total Time: 1360.029 milliseconds

Chosen Threshold: 0.88

Total Number of Similar Pairs: 83

Precision= 0.9156626506024096 , Recall= 0.7169811320754716 , Fmeasure= 0.8042328042328042

Total Time: 1371.415 milliseconds

Chosen Threshold: 0.99

Total Number of Similar Pairs: 82

Precision= 0.9146341463414634 , Recall= 0.7075471698113207 , Fmeasure= 0.7978723404255319

Note: 0.75 threshold seemly good range as 80 ish similar pairs. as threshold increase the pairs decrease lower rate 86 to 83 and to 82. best threshold range around 0.75 +- 0.03.

--Q1.3

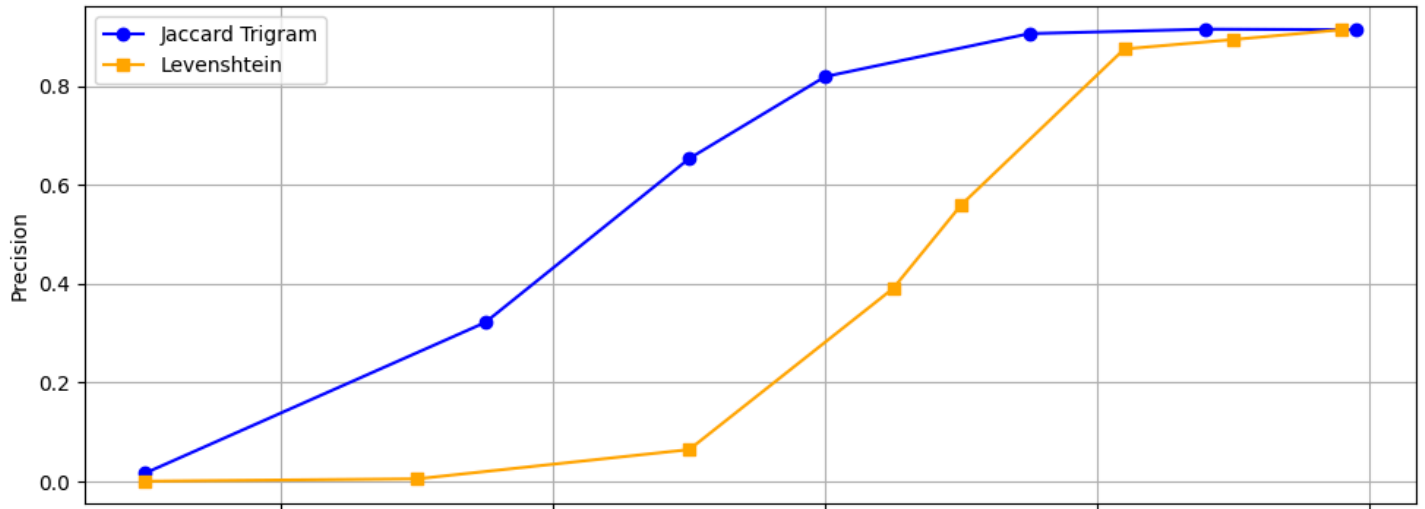
To do conclusion we must look at each component of the output.

Precision: Jaccard trigram achieves significantly higher precision at lower thresholds (eg. 0.654 at 0.5 vs. 0.0645 for Levenshtein; 0.322 at 0.35 vs. 0.0053 at 0.3). This means Jaccard identifies fewer false positives early on, which is critical for a restaurant dataset where precise matches are important to avoid incorrect pairings. However, when at high thresholds (0.88–0.99), both converge to similar precision (~0.915–0.916), but Jaccard reaches high precision (0.907) at a lower threshold (0.75) compared to Levenshtein (0.876 at 0.82).

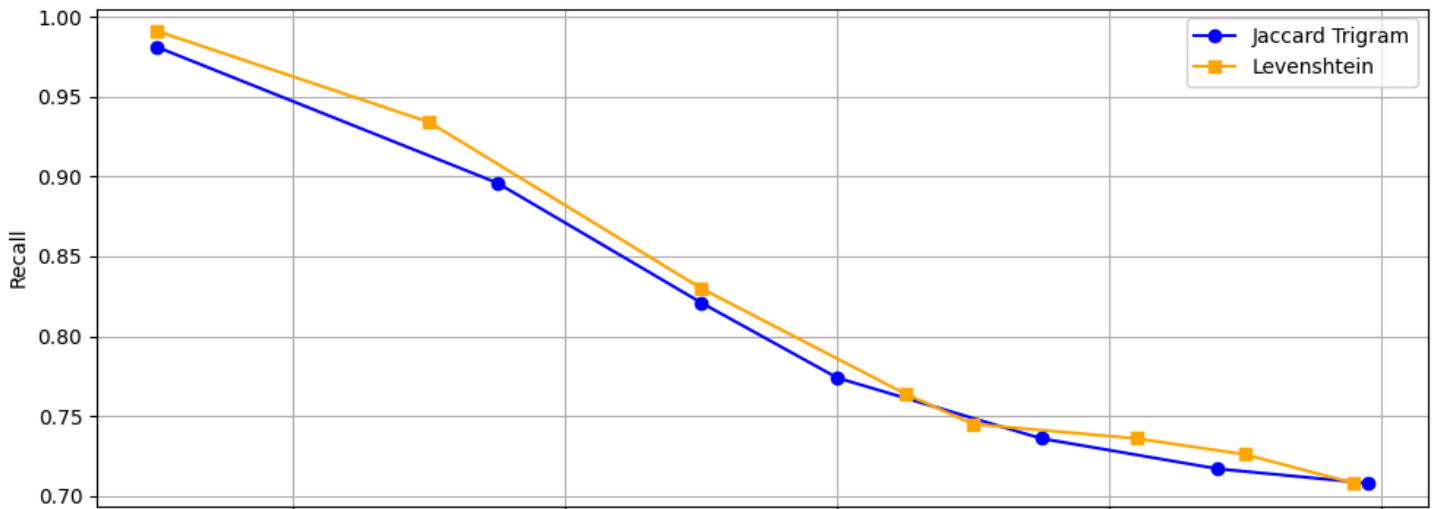
Recall: Both measures have high recall at low thresholds; 0.981 for Jaccard at 0.1 vs 0.991 for Levenshtein at 0.1, this means that both methods are ensuring most true matches are captured. But when thresholds reach higher the recall becomes comparable, at 0.708 for both at 0.98–0.99, but Jaccard maintains strong recall at its optimal F-measure.

F-measure: Jaccard trigram peaks at 0.812 (0.75), slightly better than Levenshtein's 0.802 (0.82 and 0.9). This indicates Jaccard offers a better precision-recall balance at a more practical threshold.

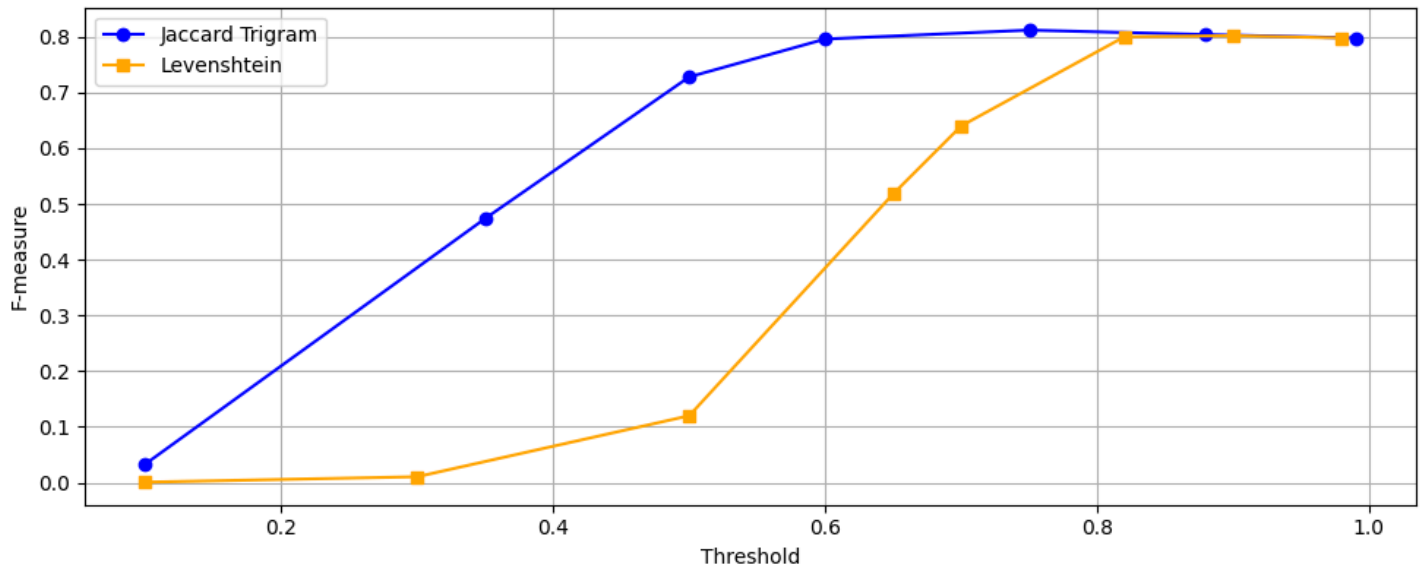
Precision vs. Threshold



Recall vs. Threshold



F-measure vs. Threshold



Observation: Jaccard trigram outperforms Levenshtein in precision at lower thresholds, which is crucial for restaurant datasets where false positives can lead to errors in applications like deduplication or recommendation systems. The slightly higher peak F-measure (0.812 vs. 0.802) suggests Jaccard is better balanced for this task.

Run time Efficiency: Jaccard trigram is significantly faster (~1.36–1.38 seconds) compared to Levenshtein (~33–34 seconds). For a big restaurant dataset, this efficiency is critical for scalability and real-time processing.

Conclusion:

Jaccard's speed makes it more practical for real world dataset which are usually big, in other words quick processing is essential for tasks like merging databases or matching user queries. Jaccard's set-based approach handles variations more effectively than Levenshtein's character-level edits.

Task 2

--Q2.1

Code:

```
--Task 2
--2.1 temp table
DROP TABLE IF EXISTS sale_temp CASCADE;

CREATE TABLE sale_temp(
    tid INT,
    sid INT,
    fname VARCHAR(20),
    lname VARCHAR(20),
    state VARCHAR(10),
    store VARCHAR(20),
    date DATE,
    pid INT,
    product VARCHAR(40),
    brand VARCHAR(40),
    unit_cost DECIMAL(10,2),
    quantity INT,
    price DECIMAL(10,2)
);

\copy sale_temp FROM '/home/s4775476/Sales.csv' DELIMITER ',' CSV HEADER QUOTE '''
```



```
-- create dimension tables
DROP TABLE IF EXISTS Staff CASCADE;

CREATE TABLE Staff (
    sid INT PRIMARY KEY,
    fname VARCHAR(20),
    lname VARCHAR(20),
    store VARCHAR(20),
    state VARCHAR(10)
);

DROP TABLE IF EXISTS Product CASCADE;

CREATE TABLE Product (
    pid INT PRIMARY KEY,
    product VARCHAR(40),
    brand VARCHAR(40)
);
```

```
41
42 DROP TABLE IF EXISTS Time_Period CASCADE;
43
44 CREATE TABLE Time_Period (
45     date DATE PRIMARY KEY,
46     month VARCHAR(10),
47     quarter VARCHAR(10),
48     year INT
49 );
50
51 --Fact Table
52 DROP TABLE IF EXISTS Sales CASCADE;
53
54 CREATE TABLE Sales (
55     tid INT,
56     sid INT REFERENCES Staff(sid),
57     pid INT REFERENCES Product(pid),
58     date DATE REFERENCES Time_Period(date),
59     quantity INT,
60     price DECIMAL(10,2),
61     unit_cost DECIMAL(10,2),
62     PRIMARY KEY (tid, sid, pid, date)
63 );
64
```

```

65  --importing Data
66  INSERT INTO Staff (sid, fname, lname, store, state)
67  SELECT DISTINCT sid, fname, lname, store, state
68  FROM sale_temp
69  WHERE sid IS NOT NULL;
70
71  INSERT INTO Product (pid, product, brand)
72  SELECT DISTINCT pid, product, brand
73  FROM sale_temp
74  WHERE pid IS NOT NULL
75  ON CONFLICT (pid) DO NOTHING;
76
77  INSERT INTO Time_Period (date, month, quarter, year)
78  SELECT DISTINCT
79      date,
80      EXTRACT(MONTH FROM date) AS month,
81      EXTRACT(QUARTER FROM date) AS quarter,
82      EXTRACT(YEAR FROM date) AS year
83  FROM sale_temp
84  WHERE date IS NOT NULL;
85
86  -- Populate Fact Table
87  INSERT INTO Sales (tid, sid, pid, date, quantity, price, unit_cost)
88  SELECT tid, sid, pid, date, quantity, price, unit_cost
89  FROM sale_temp
90  WHERE sid IS NOT NULL AND pid IS NOT NULL AND date IS NOT NULL;
91

```

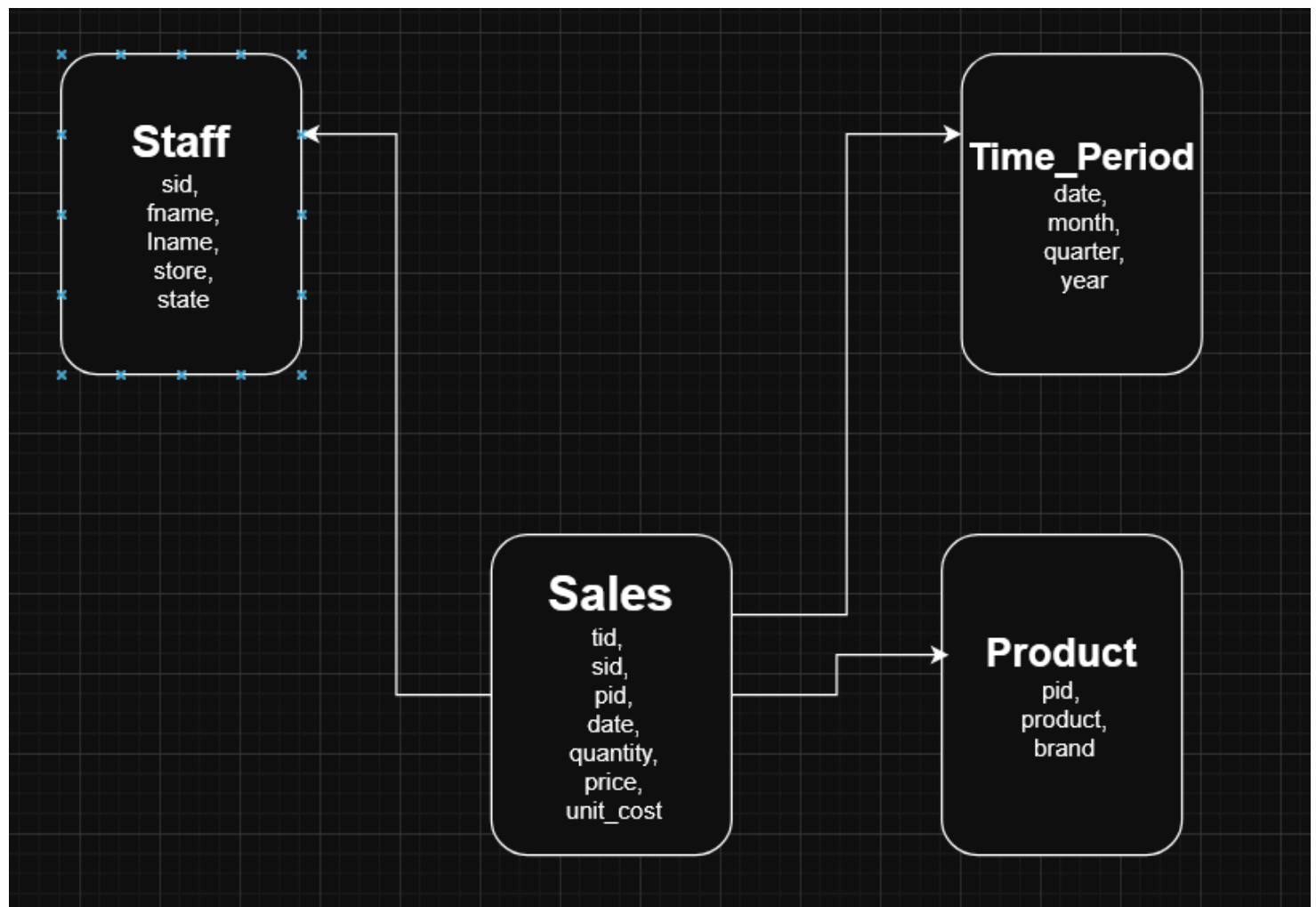
Output:

```

a2=# \i a2_s4775476.sql
DROP TABLE
CREATE TABLE
COPY 234680
psql:a2_s4775476.sql:24: NOTICE: drop cascades to constraint sales_sid_fkey on table sales
DROP TABLE
CREATE TABLE
psql:a2_s4775476.sql:34: NOTICE: drop cascades to constraint sales_pid_fkey on table sales
DROP TABLE
CREATE TABLE
psql:a2_s4775476.sql:42: NOTICE: drop cascades to constraint sales_date_fkey on table sales
DROP TABLE
CREATE TABLE
DROP TABLE
CREATE TABLE
INSERT 0 300
INSERT 0 100
INSERT 0 910
INSERT 0 234680

```

Conceptual model design:



--Q2.2

```
-- Task 2.2
\echo '2.2 part a'
SELECT COUNT(*) AS unique_staff
FROM Staff;

\echo '2.2 part b'
SELECT COUNT(*) AS transaction_count
FROM Sales s
JOIN Time_Period t ON s.date = t.date
WHERE t.year = 2022 AND t.quarter = '3';
```

Output:

```
2.2 part a
unique_staff
-----
              300
(1 row)

2.2 part b
transaction_count
-----
              24036
(1 row)
```

--Q2.3

```
114  -- Task 2.3
115
116  CREATE MATERIALIZED VIEW Sales_Time_Staff AS
117  SELECT
118      s.state, t.quarter, t.year,
119      SUM(f.quantity * (f.price - f.unit_cost)) AS total_profit
120  FROM Sales f
121  JOIN Staff s ON f.sid = s.sid
122  JOIN Time_Period t ON f.date = t.date
123  GROUP BY CUBE (s.state, t.quarter, t.year);
124
```

Output:

```
SELECT 90
```

--Q2.4

```
126 -- Task 2.4:
127 --quarter
128 CREATE VIEW profit_quarter_2021 AS
129 SELECT state, quarter, total_profit AS profit
130 FROM Sales_Time_Staff
131 WHERE year = 2021 AND STATE IS NOT NULL AND quarter is not null
132 ORDER BY state, quarter;
133
134
135 -- year
136 CREATE VIEW profit_year AS
137 SELECT state, year, total_profit AS profit
138 FROM Sales_Time_Staff
139 WHERE quarter is null And year is not null AND state is not null
140 ORDER BY state, year;
141
142 --query 2.4
143 SELECT * FROM profit_quarter_2021;
144 SELECT * FROM profit_year;
145
```

Output:

```
CREATE VIEW
CREATE VIEW
```

Profit by Quarter

state	quarter	profit
NSW	Q1	26037045.55
NSW	Q2	25667667.60
NSW	Q3	25889525.07
NSW	Q4	26117137.92
QLD	Q1	20924501.38
QLD	Q2	20842382.22
QLD	Q3	21587359.27
QLD	Q4	20937466.28
SA	Q1	19581365.01
SA	Q2	21017862.55
SA	Q3	20359894.50
SA	Q4	20446106.13
WA	Q1	20320234.71
WA	Q2	20754741.12
WA	Q3	20507141.04
WA	Q4	21030189.50
(16 rows)		

Profit by Year

state	year	profit
NSW	2021	103711376.14
NSW	2022	101622087.94
NSW	2023	51093461.84
QLD	2021	84291709.15
QLD	2022	83071386.73
QLD	2023	39502809.73
SA	2021	81405228.19
SA	2022	81780959.56
SA	2023	39806645.50
WA	2021	82612306.37
WA	2022	85057870.77
WA	2023	41190927.48
(12 rows)		

Results:

The state sales profits in each quarter of 2021				
	QLD	NSW	WA	SA
2021 Q1	20924501.38	26037045.55	20320234.71	19581365.01
2021 Q2	20842382.22	25667667.60	20754741.12	21017862.55
2021 Q3	21587359.27	25889525.07	20507141.04	20359894.50
2021 Q4	20937466.28	26117137.92	21030189.50	20446106.13

The state sales profits in each year				
	QLD	NSW	WA	SA
2021	84291709.15	103711376.14	82612306.37	81405228.19
2022	83071386.73	101622087.94	85057870.77	81780959.56
2023	39502809.73	51093461.84	41190927.48	39806645.50

--Q2.5

```

5  -- --Task 2.5
6  CREATE MATERIALIZED VIEW Sales_Product_Staff AS
7  SELECT s.store, p.product, p.brand,
8  |      SUM(f.quantity * (f.price - f.unit_cost)) AS total_profit
9  FROM Sales f
10 JOIN Staff s ON f.sid = s.sid
11 JOIN Product p ON f.pid = p.pid
12 GROUP BY CUBE (s.store, p.product, p.brand);
13

```



```

155  --top 3 part a
156  CREATE VIEW top3_profit AS
157  SELECT store, total_profit AS gross_profit
158  FROM Sales_Product_Staff
159  WHERE store is not null AND product IS NULL AND brand is null
160  ORDER BY gross_profit DESC
161  LIMIT 3;
162
163  --Most profitable part b
164  CREATE VIEW most_profitable_item_ps AS
165  SELECT DISTINCT ON (store)store, brand, product, total_profit AS most_profit
166  FROM Sales_Product_Staff
167  WHERE brand is not null and store is not null and product is not null
168  ORDER BY store, most_profit DESC;
169
170  --query 2.5
171  SELECT * FROM top3_profit;
172  SELECT * FROM most_profitable_item_ps;|

```

Output:

```
SELECT 3312
```

Top 3:

```

CREATE VIEW
store | gross_profit
-----+-----
W02   | 122705632.99
W01   | 86155471.63
S02   | 80156670.13
(3 rows)

```

Most profitable item:

Additional Information : incase if you guys want the Brand

(Side note: My only worry was what if the data was Swapped between brand and product names, therefore I do this- yeah sure it too more resource in making the cube.)

For one reason the name in brand do honestly sound like product. eg. OGtech sounds like a brand name rather than a product.

store	brand	product	most_profit
NSW01	EOS R5	OGtech	2038484.20
NSW02	EOS R5	OGtech	2451122.35
NSW03	Surface Studio 2	Tony	2583373.53
NSW04	EOS R5	OGtech	2739295.50
NSW05	EOS R5	OGtech	1894512.43
NSW06	Surface Studio 2	Tony	3185814.86
Q01	EOS R5	OGtech	3163650.98
Q02	EOS R5	OGtech	3759314.70
Q03	EOS R5	OGtech	2831220.43
Q04	Surface Studio 2	Tony	2050304.58
S01	EOS R5	OGtech	3487603.73
S02	EOS R5	OGtech	5097017.32
S03	EOS R5	OGtech	2863287.85
W01	EOS R5	OGtech	4846956.47
W02	EOS R5	OGtech	6643037.81
(15 rows)			